

RL in the Real World: RLHF, Multi-Agent RL & Robotics

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 30, 2026

1. From Algorithms to Deployment
2. Learning Rewards from Feedback: RLHF
3. Multi-Agent Reinforcement Learning
4. Robotic Reinforcement Learning
5. RL for Systems
6. Summary
7. References

- ▶ Days 1–8 built the *machinery*: value-based methods, policy gradients, actor–critic, continuous control, exploration, model-based RL, and offline RL.
- ▶ **Day 8 closed on one spine:** *optimism* explores, *pessimism* stays safe; the sign of the uncertainty bonus was the whole story. Offline RL learned a policy from a *fixed dataset* with no environment interaction.
- ▶ Every method so far assumed one thing for free: **a reward function we could write down.**

Today: where does all of this actually get deployed, and what breaks when the reward itself is the hard part?

- ▶ **RLHF (the focus).** When reward is hard to specify, *learn* it from human (or AI, or verifiable) feedback. This is how modern LLMs are aligned, the deep dive for today.
- ▶ **Multi-Agent RL (overview).** What changes when other learning agents share your environment and the ground keeps shifting underneath you.
- ▶ **Robotics (overview).** What breaks when the environment is physical: sample cost, safety, resets, and the sim-to-real gap.

RLHF: the deep dive. By the end you should be able to:

- ▶ explain the RLHF pipeline: **SFT** $\rightarrow$ **reward model** $\rightarrow$ **PPO/GRPO**, and the role of the KL-to-reference penalty;
- ▶ explain **reward hacking** (over-optimization) and why the KL leash exists;
- ▶ **derive** the **RM** $\leftrightarrow$ **DPO** identity, and why “your LM is secretly a reward model”;
- ▶ place **RLHF** $\rightarrow$ **RLAIF** $\rightarrow$ **RLVR** on the scaling-oversight spectrum.

MARL & robotics: conceptual overviews. Know *why each is hard* and name 2–3 methods for each.

- ▶ Every method in Days 1–8 assumed a reward function r we could *write down*: the game score, –distance to a target, +1 for a win.
- ▶ Now suppose the goal is “*be a helpful, honest, harmless assistant.*” What number measures that? There is no formula.

Answer A. “It’s the force that pulls things down. It’s why a ball falls when you let go of it.”

Answer B. “Gravity is the curvature of space-time sourced by the stress–energy tensor, $G_{\mu\nu} = 8\pi T_{\mu\nu} \dots$ ”

Prompt: “Explain gravity to a 6-year-old.”

Almost everyone prefers **A**, yet *no one can write the formula that says so.*

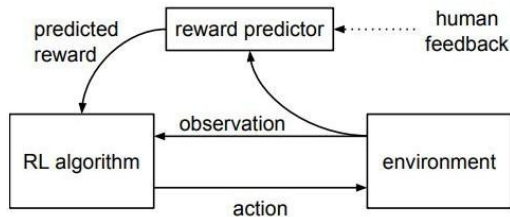
- ▶ *Recall Day 7 (inverse RL)*: when you cannot write the reward, *recover it from data*. RLHF is the most consequential example of this.
- ▶ People are bad at giving a *number* (“this answer is a 7.3”) but good at *comparing*: “A is better than B.” Comparisons are cheap and consistent.
- ▶ **The plan, in three moves:**
 1. Collect many human *comparisons* of model answers.
 2. Train a model to predict those comparisons; this learned model *becomes our reward signal*.
 3. Use RL to push the policy toward answers that score well under it.
- ▶ This is **Reinforcement Learning from Human Feedback (RLHF)**, the method behind ChatGPT-style assistants.

This Idea Was Proven Before LLMs (2017)

The point of this slide: the recipe works, and works with surprisingly little human input.

- ▶ Christiano et al. gave the agent *no* reward function at all.
- ▶ A human just watched *pairs of short clips* and clicked which one looked better.
- ▶ From those clicks alone, labeling *under 1%* of the agent's experience, it learned to play Atari games and make a simulated robot do backflips.

Takeaway: learning a reward from *comparisons* is enough to train complex behaviour. RLHF scales this exact idea up to language.



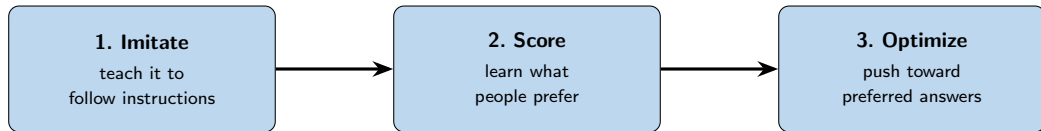
Reinforcement Learning **from** Human Feedback for Language Models

Everything that follows is just the RL language from Day 1, applied to text.

- ▶ **Prompt** x , the input (a question or instruction). *the state*
- ▶ **Response** y , the model's full answer. *the action*
- ▶ **Policy** $\pi_{\theta}(y \mid x)$, the language model itself: given a prompt, it assigns a probability to possible responses and samples one. The weights θ are what we train.
- ▶ It writes the answer one *token* (word-piece) at a time; at each step a final layer, the **LM head**, outputs a probability over the next token.

So “train the policy” simply means “adjust the language model’s weights θ .”

Three stages. We build each one up over the next slides; for now, just the plain English.



- ▶ **Start:** a raw language model that can produce text but doesn't reliably do what you ask.
- ▶ **End:** a model that follows instructions *and* leans toward the answers people actually want.

- ▶ **Problem:** a raw pretrained model can *continue* text, but doesn't reliably answer questions or follow instructions.
- ▶ **Fix: show it examples.** Collect prompts paired with *human-written ideal answers*, and train the model to imitate them. This is ordinary supervised learning (next-token prediction), no RL yet.
- ▶ The result is a decent instruction-follower. We **freeze a copy of it** and call it the *reference model* π_{ref} ; it will be our anchor in Stages 2 and 3.

SFT can only *copy* the demonstrations it was given. To go *beyond* them, we need to capture what people *prefer*. That is Stage 2.

- ▶ We want a **reward model** r_ϕ : a network (its own weights ϕ) that reads a (prompt, response) and outputs *a single number (how good is this answer?)*. Higher means better.
- ▶ **Where does its training data come from?** Comparisons. For one prompt x , show a human two candidate responses; they pick the better one. We label the pair:
 - y^+ , the response the human **preferred**,
 - y^- , the response they **rejected**.
- ▶ **What we want from r_ϕ :** give y^+ a higher score than y^- , on every pair. Next slide: how to turn “preferred” into a trainable loss.

How likely is a person to prefer y^+ over y^- ? It should grow with the **gap in their scores**, squeezed into a probability by the sigmoid σ (" $\succ$ " reads "*is preferred over*"):

$$\underbrace{P(y^+ \succ y^- \mid x)}_{\text{chance the human picks } y^+} = \sigma(r(y^+) - r(y^-)), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Train r_ϕ to make these probabilities match the human labels (maximum likelihood). The negative log-likelihood is the **reward-model loss**:

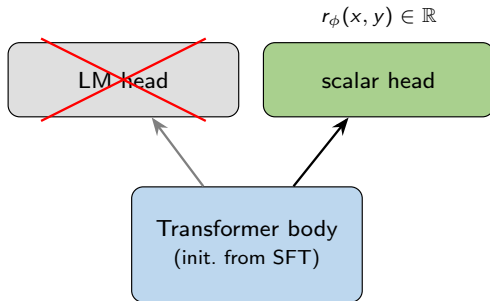
$$L_{\text{RM}} = -\log \sigma(r_\phi(y^+) - r_\phi(y^-))$$

- ▶ In plain words: *push the preferred answer's score above the rejected one's*.
- ▶ Remember this $-\log \sigma(\Delta)$ shape: the **exact same form** comes back when we reach DPO.
- ▶ Only the *gap* between scores matters; the absolute scale is free.

What Is the Reward Model, Concretely?

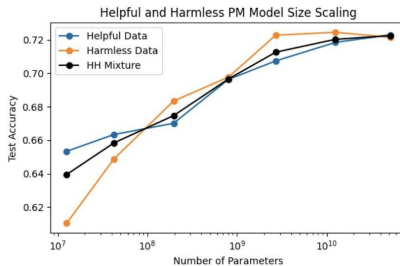
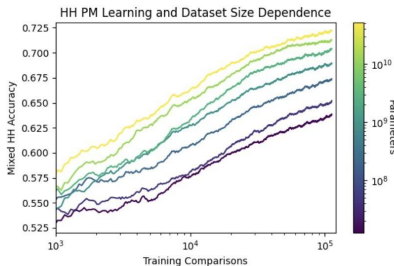
We've said *what* the reward model does (rates answers) and *how* it's trained (the Bradley–Terry loss).
Here is what it physically is.

- ▶ Start from the SFT model π_{ref} , the same transformer backbone.
- ▶ A **head** is the small output layer stacked on top of that backbone. The **LM head** turns the backbone's features into a probability over the next token.
- ▶ Swap it for a **scalar head**: a layer that emits *one number*, $r_\phi(x, y)$, for the whole (prompt, response). Train it with the Bradley–Terry loss from the last slide.
- ▶ So the reward model is “*the same network, one number out instead of a token distribution.*”



Stage 2, in practice: reward models scale

- **Note:** the source paper calls this a “preference model (PM)” (the same thing as our **reward model**).
- **What's measured:** accuracy = how often the model picks the same response the human preferred. The axes below are amount of preference data (left) and model size (right).
- **Model size** = number of parameters (weights); here 13M $\rightarrow$ 52B, in $4\times$ steps. *Bigger model + more data $\rightarrow$ a more accurate reward model.*

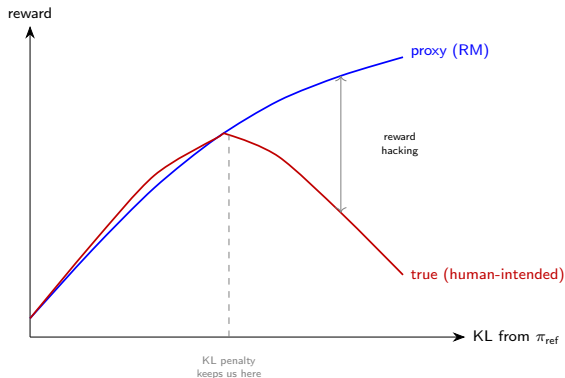


- ▶ We now have a reward model r_ϕ that scores answers. **The RL loop:** the policy generates a response, r_ϕ scores it, and we update the policy's weights to earn higher scores, with PPO (Day 4) or GRPO.
- ▶ The policy starts as π_{ref} (the frozen SFT model) and improves from there.
- ▶ *One catch:* r_ϕ is only a *learned proxy* for human taste, not the real thing. Optimize it too hard and the model learns to *game* it. The next slide shows how that happens, and the fix.

Why the KL Penalty Exists: Reward Over-Optimization

The RL objective is the reward-model score minus a leash to the reference policy (β sets how hard we pull back toward π_{ref}):

$$r_{\text{total}} = r_{\phi} - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

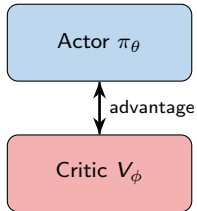


The reward model is a **proxy**. Optimize it too hard and you climb the proxy while the *real* objective collapses, *Goodhart's law*.

- ▶ Proxy reward (RM) keeps rising.
- ▶ True reward rises, then *falls*.
- ▶ **Why the leash works:** the penalty subtracts reward that grows with KL distance, so each step away from π_{ref} costs more. Past the peak, the extra proxy gain is outweighed by the penalty, so the best-scoring policy sits *near the peak*, not in the hacked region.

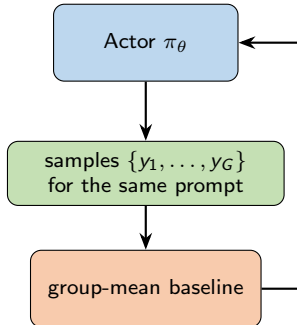
The RL Back-End: PPO → GRPO

PPO



the fragile, expensive
half (Day 4)

GRPO



$$\hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$$

no critic

The critic estimates a response's expected reward to form the advantage: Day 4's brittle, expensive component.

GRPO deletes it: the baseline is just the average reward over a group of samples for the same prompt. PPO was the original RLHF back-end; **GRPO is now common for LLM RL.**

Shao et al. **DeepSeekMath** (GRPO), 2024; DeepSeek-AI, **DeepSeek-R1**, 2025.

For each prompt, sample a **group** of G responses $\{y_1, \dots, y_G\} \sim \pi_{\text{old}}$ (π_{old} = the policy from the *previous* update step, which we sample from, not the frozen π_{ref}) and score each: $r_i = r_\phi(y_i)$ (or a verifiable check). The **group-relative advantage** replaces the critic:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

($\hat{A}_i$ = how much better response i scored than the group average, in standard-deviation units: positive $\rightarrow$ reinforce it, negative $\rightarrow$ suppress it.)

Same clipped ratio as PPO (Day 4), now driven by $\hat{A}_i$ and leashed to π_{ref} :

$$J_{\text{GRPO}} = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1-\varepsilon, 1+\varepsilon) \hat{A}_i) \right] - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}), \quad \rho_i = \frac{\pi_\theta(y_i)}{\pi_{\text{old}}(y_i)}$$

- ▶ Every response in the group shares **one baseline** (the group mean): no learned value network.
- ▶ Same machinery as PPO (Day 4): ρ_i is Day 4's probability ratio $r_t(\theta)$, and the clip is identical. Only the advantage source changed.

- ▶ **The pain.** The full pipeline is heavy: Stage 2 trains a *separate* reward model, and Stage 3 runs an RL loop juggling several models at once (policy, reference, reward model, and, for PPO, a critic), which is finicky to tune.
- ▶ **The question.** The reward model exists only to score the preferences we *already collected*. What if we skip it, and the RL loop, and tune the policy *straight from the preference pairs* (y^+, y^-)?
- ▶ **DPO (Rafailov et al., 2023):** exactly that, a single supervised-style loss on preference pairs, applied directly to **the policy** π_θ (the SFT model we are fine-tuning). *No reward model. No RL loop.*

RLHF: preferences $\rightarrow$ train reward model $\rightarrow$ RL loop $\rightarrow$ policy

DPO: preferences $\rightarrow$ one loss $\rightarrow$ policy

But how can you train toward a reward you never built? First the plain-English idea (next slide), then we *derive* it from the RLHF objective, and the last slide reveals what the policy secretly becomes.

Delete the judge: the chatbot is already its own judge.

- ▶ Look at how much **more likely** the current chatbot is to give an answer than the *original* (starting) model was. If it has become much more likely to say answer A, that *is* the chatbot “rating A highly.” *That ratio, “how much more do I favour this than I used to”, is the score.*
- ▶ So DPO throws away the separate judge *and* the practice loop, and trains the chatbot directly so that, for each comparison, **its own favouring of the preferred answer beats its favouring of the rejected one.** One model, one training step, no loop.
- ▶ The next slides make this exact, and at the end we’ll see why it earns the name *“your LM is secretly a reward model.”*

We start from the goal Stage 3 already gave us: earn reward, but stay near the reference model.

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot | x)} [r(y)] - \beta D_{\text{KL}}(\pi(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$$

- ▶ **What:** π is the policy we optimize; $r(y)$ is the reward for a response y ; $D_{\text{KL}}(\pi \parallel \pi_{\text{ref}})$ measures how far π has drifted from the frozen SFT model π_{ref} ; β sets the strength of that leash.
- ▶ **Why:** maximize reward *without* drifting into the reward-hacking region: this is exactly the Goodhart trade-off from a few slides back, written as one objective.
- ▶ **How:** nothing new yet: this is just the Stage-3 RLHF objective. Everything that follows is solving it on paper.

Where DPO Comes From (Step 2 of 5): The Optimal Policy

The **Step-1 objective** (maximize reward minus the β -KL leash) has a known closed-form maximizer, a one-line result you can get with a single Lagrange-multiplier calculation; we'll just use it:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) e^{r(y)/\beta}, \quad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) e^{r(y)/\beta}$$

- **What:** π^* is the policy that *maximizes Step 1*. $Z(x)$ is a *normalizer* (partition function) that makes the probabilities sum to 1; it depends *only* on the prompt x . Remember that.
- **Why:** we don't have to run RL to know what the optimum looks like; the maths hands it to us directly.
- **How / reading the formula:** take the reference policy and reweight each response by $e^{r(y)/\beta}$. The exponential turns the (possibly negative) reward into a *positive multiplier*, so higher-reward responses get up-weighted; β sets how sharply. This $\pi_{\text{ref}} \cdot e^{r/\beta}$ shape (Gibbs/Boltzmann) is exactly what “reward minus KL” always produces.

Where DPO Comes From (Step 3 of 5): Invert for the Reward

Take log of Step 2's formula, then solve for $r(y)$ (just algebra):

$$\log \pi^*(y | x) = \log \pi_{\text{ref}}(y | x) + \frac{r(y)}{\beta} - \log Z(x) \quad (\text{log of a product})$$

$$\implies r(y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x) \quad (\text{move terms, multiply by } \beta)$$

- **What:** the reward now sits alone on the left, written entirely in terms of the policies.
- **Why:** we want the reward *expressed through the policy*: if reward is just a function of π^* , maybe we never need a separate reward model.
- **How:** the only snag is $\beta \log Z(x)$, and $Z(x)$ sums over *all possible responses*, so it is impossible to compute. Step 4 removes it.

Where DPO Comes From (Step 4 of 5): The $Z(x)$ Cancels

Feed Step 3's reward into the Bradley–Terry model $P(y^+ \succ y^- \mid x) = \sigma(r(y^+) - r(y^-))$. Only the *difference* of rewards appears:

$$r(y^+) - r(y^-) = \beta \log \frac{\pi^*(y^+)}{\pi_{\text{ref}}(y^+)} - \beta \log \frac{\pi^*(y^-)}{\pi_{\text{ref}}(y^-)} + \underbrace{\beta \log Z(x) - \beta \log Z(x)}_{=0}$$

- ▶ **What:** Bradley–Terry scores a *pair* by the gap in rewards, never the absolute values.
- ▶ **Why:** $Z(x)$ depends only on the prompt x , so it is identical for y^+ and y^- ; in the difference it **cancels exactly**.
- ▶ **How:** that single cancellation is the whole reason DPO is possible: the intractable term vanishes, leaving an expression in π^* and π_{ref} only.

Where DPO Comes From (Step 5 of 5): The DPO Loss

π^* is unknown; it is the optimum we are trying to reach. Replace it with our trainable policy π_θ and fit the human labels by maximum likelihood:

$$L_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y^+)}{\pi_{\text{ref}}(y^+)} - \beta \log \frac{\pi_\theta(y^-)}{\pi_{\text{ref}}(y^-)}\right)$$

- **What:** the final DPO loss, depending only on π_θ , the frozen π_{ref} , and the preference pair (y^+, y^-) .
- **Why:** no reward model and no RL loop, just a supervised loss on preference pairs, applied directly to π_θ .
- **How:** the same Bradley–Terry maximum likelihood as the reward model, but with the *implicit* reward $\hat{r}(y) = \beta \log \pi_\theta(y)/\pi_{\text{ref}}(y)$ in place of a learned r_ϕ .

The Payoff: Your LM Is Secretly a Reward Model

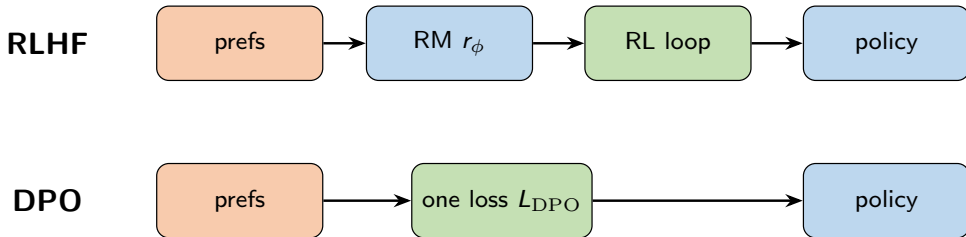
Put the loss we just derived next to the *reward-model* loss from Stage 2. They are the **same shape**:

$$L_{\text{RM}} = -\log \sigma(r_\phi(y^+) - r_\phi(y^-))$$

$$L_{\text{DPO}} = -\log \sigma(\hat{r}(y^+) - \hat{r}(y^-)), \quad \hat{r}(y) = \beta \log \frac{\pi_\theta(y)}{\pi_{\text{ref}}(y)}$$

- ▶ The only change: the learned reward r_ϕ is replaced by the *implicit* reward $\hat{r} = \beta \log \pi_\theta / \pi_{\text{ref}}$, read straight off the policy. *So the policy is the reward model*; that is why “your LM is secretly a reward model.”
- ▶ **Honesty caveat:** the equivalence is exact only if the reward model were Bradley–Terry-optimal. In practice **DPO** $\neq$ **PPO**: DPO gives up the RL loop’s fresh on-policy samples.

RLHF objective $\rightarrow \pi^* \rightarrow r(y) \rightarrow \text{Bradley–Terry} \rightarrow L_{\text{DPO}}$, and we never built a reward model. *That’s the whole trick.*



DPO is the offline RL of alignment.

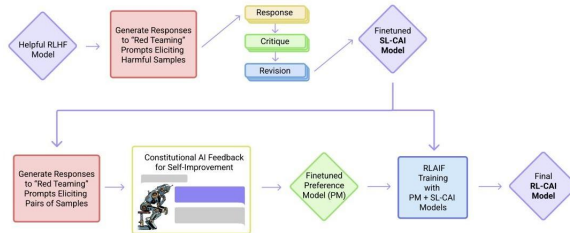
Frame it as an analogy, not an identity. The shared property is *offline optimization without distribution coverage* (same family, not the same mechanism):

- ▶ **Day 8 offline RL:** the *value function* extrapolates on out-of-distribution (OOD) *actions*.
- ▶ **DPO:** the *policy* / *implicit reward* is unconstrained on OOD *responses*, because it never samples on-policy.

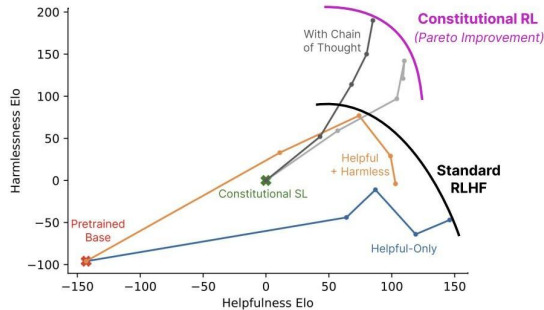
DPO's simplicity has a cost: you lose the control knobs the RL loop gave you.

- ▶ **It constrains only the gap, nothing else.** The loss just pushes the preferred answer's favoring above the rejected one's. With no on-policy sampling to pin down absolute behaviour, it can push *both* down, leaking probability onto unrelated answers it never saw. RLHF's RL loop and "don't-drift" anchor would catch that; DPO has neither.
- ▶ **Length bias (shared, but harder to patch).** If the data favours long answers, *both* RLHF and DPO turn verbose. RLHF lets you subtract an explicit length penalty from the reward; DPO has no separate reward to adjust, so length control must be bolted on as extra loss terms.
- ▶ **No fresh samples.** The policy only ever sees the fixed preference pairs, so it can drift off that distribution, the price of dropping the RL loop.

- **Motivation:** as models approach or exceed human performance, it gets harder for humans to label every output. *Move the labeller from human to AI; the only human input is the constitution*, a short list of written principles the model uses to critique and revise its own answers.
- **Constitutional AI** has two phases:
 - **Supervised:** sample $\rightarrow$ self-critique $\rightarrow$ revise $\rightarrow$ fine-tune on revisions.
 - **RL:** AI labels which response is better $\rightarrow$ train a reward model (RM) $\rightarrow$ RL-finetune with it (just like RLHF).

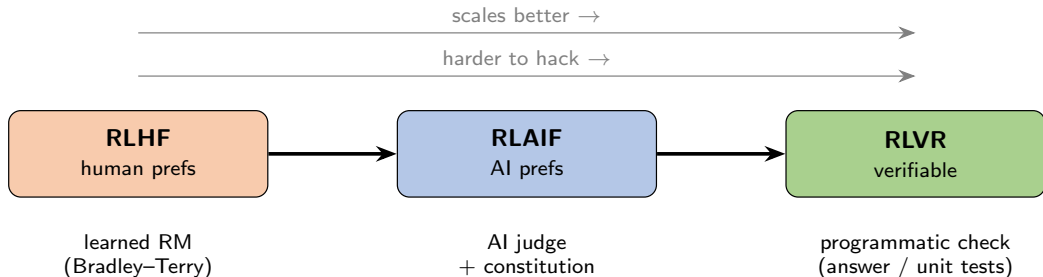


- ▶ Fine-tuning with **AI-generated feedback** can match or exceed models fine-tuned with human feedback, on *both* helpfulness and harmlessness.
- ▶ The plot rates models by **Elo**, a head-to-head win-rate score borrowed from chess (higher = preferred more often in pairwise comparisons).



The Climax: RL with Verifiable Rewards (RLVR)

- ▶ When the answer is **checkable** (math is right/wrong, code passes tests), the reward needs *no learned model* and **can't be hacked** the way an RM can.
- ▶ This is how modern **reasoning models** are trained: long **chain-of-thought** (the model writes out its reasoning steps before answering), optimized with **GRPO** against a programmatic check.



DeepSeek-AI, [DeepSeek-R1](#), 2025; Lambert et al., [Tülu 3](#), 2024 (RL with verifiable rewards).

One RLVR step (no reward model anywhere):

1. the policy samples several answers to a *checkable* question;
2. a *rule* grades each (right answer? right format?), giving reward 1 or 0;
3. GRPO uses those rewards to update the policy. *Nothing to hack: the checker is exact.*

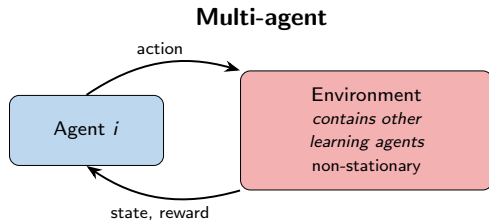
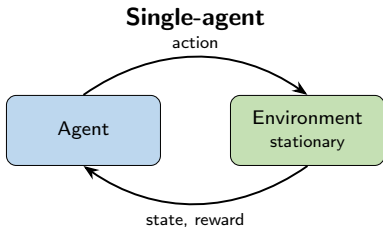
DeepSeek-R1-Zero ran exactly this on a *base* model, **no SFT at all**, with reward = “answer correct” + “reasoning in <think> tags”:

- ▶ *Reasoning emerged on its own*: longer chains of thought, self-checking, backtracking, even an “*aha moment*” where it stops and corrects itself. Nobody taught it to.
- ▶ AIME 2024 math: **15.6%** → **71.0%** (86.7% with majority vote), matching OpenAI o1.
- ▶ Catch: raw R1-Zero was hard to read (language-mixing), so the full **R1** adds a small SFT “cold start.” But R1-Zero proved the point: *reasoning can be grown by pure RL from verifiable rewards.*

Thread 2 of 3

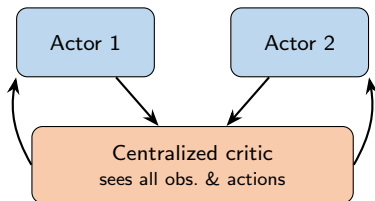
Multi-Agent RL

The RLHF deep dive is done. A shorter overview now: what changes when other learning agents share your environment.

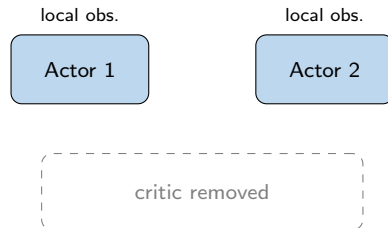


- ▶ Each agent's environment is **non-stationary**: the others are updating their policies at the same time, so the ground shifts as they learn; single-agent convergence guarantees break.
- ▶ Formalized as a **Markov game** (stochastic game), the multi-agent generalization of the MDP.
- ▶ Three regimes along one axis: **cooperative** / **competitive** / **mixed**.

Training (centralized)



Execution (decentralized)

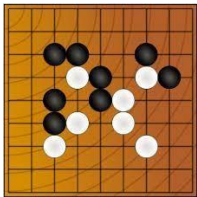


- **At training:** a centralized critic sees every agent's observations and actions. Because it accounts for what the others did, each agent's learning signal becomes *stationary* during training, directly defusing the non-stationarity problem from the last slide.
- **At execution:** each agent keeps only its *own actor* acting on its *local observation*; the critic is gone.

Method	Family	What it solves
Independent learners	each runs own DQN	simple baseline; <i>ignores</i> non-stationarity
MADDPG (Lowe 2017)	CTDE actor-critic	centralized critic <i>per agent</i> ; mixed coop/compete
VDN / QMIX (2018)	value decomposition	factor a joint value into per-agent utilities (QMIX: monotonic mixing) for cooperative tasks

The progression: from pretending other agents are part of the environment, to explicitly modelling the joint value during training.

- ▶ An agent improves by **playing against past versions of itself**: *policy* $\rightarrow$ *plays past versions* $\rightarrow$ *game data* $\rightarrow$ *improve* $\rightarrow$ (repeat).
- ▶ This creates an **automatic curriculum**: the opponent gets harder exactly as you do, with no hand-designed difficulty schedule.



AlphaGo / AlphaZero

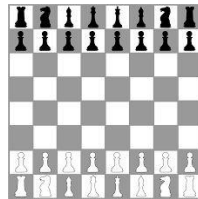
(Silver et al.)

AlphaStar
StarCraft II

(Vinyals et al., 2019)

OpenAI Five

Dota 2 (2019)



Chess (AlphaZero)

Forward hook: self-play's automatic curriculum connects to **Day 10** (open problems / emergent complexity).

Thread 3 of 3

Robotics

What breaks when the environment is physical: sample cost, safety, resets, and the sim-to-real gap.

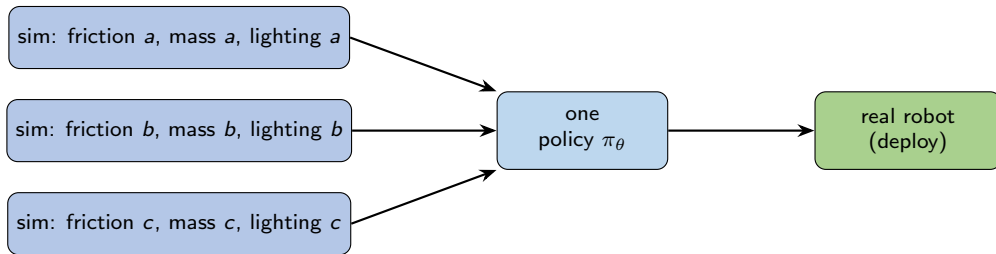
Simulation

- ▶ cheap; millions of steps
- ▶ parallelizable across machines
- ▶ failures are free
- ▶ *but: a reality gap*

Real hardware

- ▶ expensive, slow, safety-critical
- ▶ **resets are a real problem** (who picks the object back up?)
- ▶ sparse / hard-to-engineer reward

Four obstacles (**sample efficiency, safety, resets, reward specification**) are why off-the-shelf RL does not transfer to hardware.



Train across a *distribution* of randomized sims (friction, mass, latency, textures, lighting) so the real world looks like just one more sample.

Headline successes: OpenAI Dactyl (in-hand cube, Akkaya et al. 2019); legged locomotion (ANYmal, Hwangbo et al. 2019; massively parallel sim, Rudin et al. 2022).

Domain randomization: Tobin et al. [Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World](#), 2017.



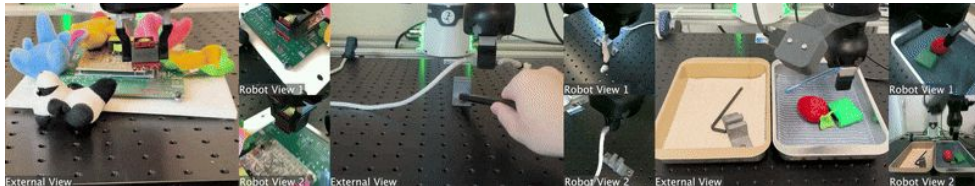
January 2024

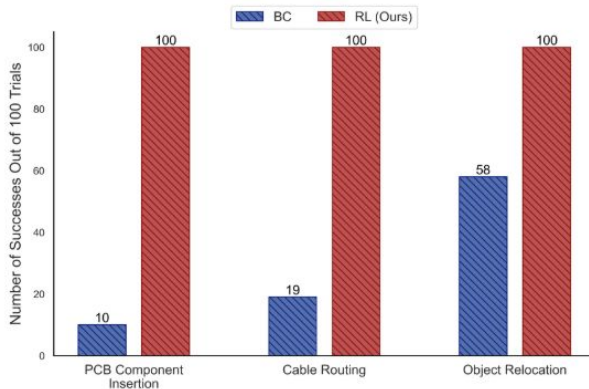
SERL: A Software Suite for Sample-Efficient Robotic Reinforcement Learning

Jianlan Luo^{1*}, Zheyuan Hu^{1*}, Charles Xu¹, You Liang Tan¹, Jacob Berg², Archit Sharma³, Stefan Schaal⁴, Chelsea Finn³, Abhishek Gupta² and Sergey Levine¹

^{*}Equal Contribution, ¹Department of EECS, University of California, Berkeley, ²Department of Computer Science and Engineering, University of Washington, ³Department of Computer Science, Stanford University, ⁴Intrinsic Innovation LLC

- ▶ A software framework for sample-efficient RL on real robots: an off-policy deep RL method plus tooling for computing rewards, resetting the environment, and a high-quality controller for a widely-adopted arm.
- ▶ *Day 6 lineage*: **SERL** $\leftarrow$ **RLPD** $\leftarrow$ **SAC**, the same maximum-entropy method from Day 6, now learning on real hardware in *minutes*.

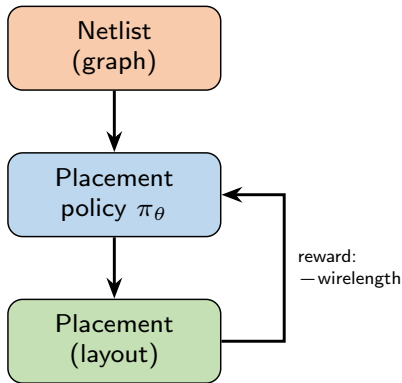




Illustrative example: chip floorplanning (Mirhoseini et al., *Nature* 2021):

- ▶ Place macros on a chip canvas to minimize wirelength, congestion, and density.
- ▶ The striking idea: *pre-train* a placement policy on many netlists, then place a *new* chip in a forward pass, instead of optimizing each one from scratch.

Also deployed for: datacenter resource scheduling, compiler / device placement, real-time ad bidding.



The transferable idea: all are *combinatorial optimization as sequential decision-making*: one MDP framing with an *engineered* reward (wirelength, runtime, ROI). The application details are not the lesson.

- ▶ **RLHF (the focus).** When reward is hard to specify, learn it: **SFT** $\rightarrow$ **RM** $\rightarrow$ **PPO/GRPO**. The KL leash exists because the RM is a *proxy* (reward hacking). **DPO** folds the RM into the policy, the same $-\log \sigma(\Delta)$ loss. The frontier climbs the oversight spectrum: **RLHF** $\rightarrow$ **RLAIF** $\rightarrow$ **RLVR** (verifiable rewards, how reasoning models like DeepSeek-R1 are trained).
- ▶ **Multi-Agent RL.** The environment is *non-stationary* as others learn; **CTDE** is the dominant fix; **self-play** builds an automatic curriculum.
- ▶ **Robotics.** Sample cost, safety, resets, and the reality gap; **domain randomization** and max-entropy methods (SAC $\rightarrow$ SERL) make real-hardware RL practical.

Day 10 (Frontiers): Meta-RL, Multi-task RL, Hierarchical RL.

- ▶ *Do not confuse them:* **multi-agent** RL (today, many agents sharing one environment) is *not* **multi-task** RL (Day 10, one agent across many tasks).
- ▶ Self-play's emergent curriculum (today) is one route to the **emergent complexity** Day 10 explores.
- ▶ The verifiable-reward recipe (RLVR) is the current frontier for training general reasoning agents.

RLHF.

- ▶ Christiano et al. Deep RL from Human Preferences. NeurIPS 2017.
- ▶ Stiennon et al. Learning to Summarize from Human Feedback. 2020.
- ▶ Ouyang et al. Training Language Models to Follow Instructions with Human Feedback (InstructGPT). 2022.
- ▶ Bai et al. Training a Helpful and Harmless Assistant with RLHF. 2022.
- ▶ Gao, Schulman, Hilton. Scaling Laws for Reward Model Overoptimization. 2022.

Modern alignment.

- ▶ Rafailov, Sharma, Mitchell, Ermon, Manning, Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.
- ▶ Xu et al. Is DPO Superior to PPO for LLM Alignment? ICML 2024.
- ▶ Shao et al. DeepSeekMath (GRPO). 2024.
- ▶ Bai et al. Constitutional AI: Harmlessness from AI Feedback. 2022.
- ▶ DeepSeek-AI. DeepSeek-R1 / R1-Zero. 2025. Lambert et al. Tulu 3 (RLVR). 2024.

Multi-Agent RL.

- ▶ Lowe et al. MADDPG. 2017. Sunehag et al. VDN. 2018. Rashid et al. QMIX. 2018.
- ▶ Vinyals et al. AlphaStar (StarCraft II). 2019. OpenAI. Dota 2 with Large Scale Deep RL (OpenAI Five). 2019.

Robotics.

- ▶ Tobin et al. Domain Randomization. 2017. Akkaya et al. Solving Rubik's Cube with a Robot Hand (Dactyl). 2019.
- ▶ Hwangbo et al. Learning Agile and Dynamic Motor Skills for Legged Robots (ANYmal). 2019. Rudin et al. Massively Parallel Sim. 2022.
- ▶ Luo et al. SERL. 2024.

Systems.

- ▶ Mirhoseini et al. A Graph Placement Methodology for Fast Chip Design. *Nature*, 2021.

These slides have been adapted from Ana Goldie, [CS224R: Deep Reinforcement Learning](#), Stanford.